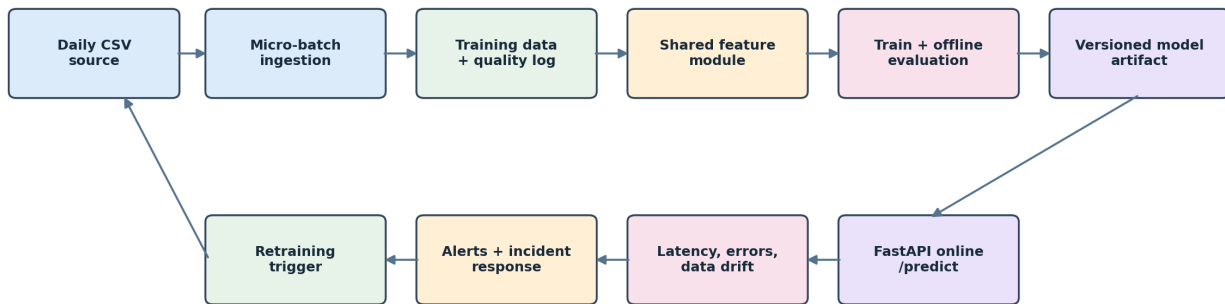


Design and Build a Mini Production ML System

Customer Churn Prediction

Customer Churn Production ML Architecture



Offline path: ingestion to promotion | Online path: prediction to monitoring | Feedback closes the retraining loop

Student: Prakash Babu M. | Individual submission | Model version: churn-rf-1.0.0

Executive summary

This project implements a compact production ML system that estimates the probability that a telecom customer will cancel service. The system uses the public IBM Telco Customer Churn dataset containing 7,043 customer records. It covers the lifecycle from batch ingestion and shared feature computation through reproducible training, offline evaluation, online serving, latency measurement, data-quality monitoring, and policy-based retraining. The objective is not state-of-the-art accuracy; it is a credible, testable design whose operational choices are explicit.

A class-balanced logistic regression provides the baseline and a tuned random forest provides the candidate. Both use the same held-out test split and preprocessing contract. The candidate reached ROC-AUC 0.8457, accuracy 0.7644, and F1 0.6360. It passed the promotion rule and is exposed through a FastAPI /predict endpoint. A 250-request local benchmark averaged 39.994 ms. Automated tests cover feature logic, API output, idempotent ingestion, and retraining triggers.

Repository map

Component	Implementation	Input	Output	Control
Training	scripts/train.py	Raw CSV	Model + JSON	Seeded split
Serving	FastAPI /predict	Customer JSON	Probability	Schema validation
Monitoring	check_drift.py	Recent CSV	Drift JSON	Threshold alerts
Tests	pytest	Code + model	Pass/fail	4 checks

1. Problem, data and feature design

Business problem

Retention teams need to identify customers likely to cancel service. The model scores one active customer and labels probabilities at or above 0.50 as churn. It prioritizes a limited outreach budget, not automatic pricing or service decisions. False positives waste incentives; false negatives lose recurring revenue.

Data source and assumptions

The IBM Telco Customer Churn sample has 7,043 records covering demographics, tenure, services, contract, payment, charges, and the Churn label. Churn is mapped from Yes/No to 1/0. customerID supports deduplication but is excluded from modeling. Eleven blank TotalCharges values are replaced with MonthlyCharges times tenure. The split is stratified because 26.54% churned. This historical sample is not evidence of current telecom performance.

Feature construction

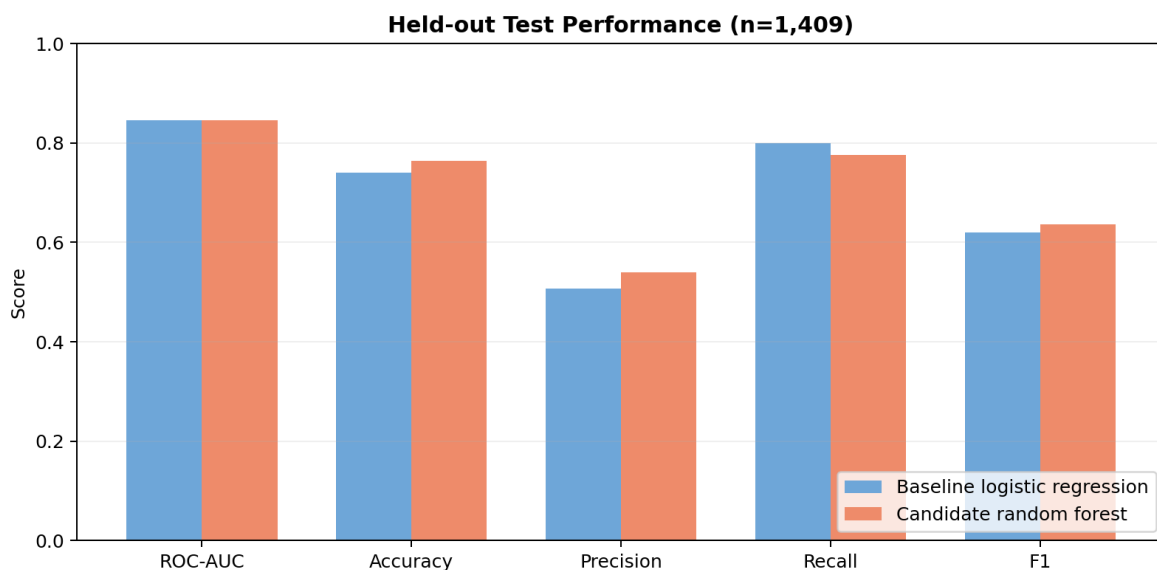
Nine engineered features represent business hypotheses: average charge per month, current-versus-historical charge gap, tenure in years, new-customer indicator, month-to-month indicator, automatic-payment indicator, counts of support and streaming services, and estimated lifetime value. All are inexpensive and deterministic.

Offline, online and skew control

Inputs are available from the customer profile. At scale, cumulative charges and tenure would be materialized daily and looked up by customerID. Deterministic feature definitions live in one shared module called by training and API inference, preventing duplicate logic. The fitted pipeline handles numeric gaps and unseen categories; API ranges reject impossible values.

Micro-batch ingestion

The ingestion script merges a daily CSV, keeps the latest duplicate customerID, and logs timestamp, source, rows read, and rows after merge. This provides idempotence and an audit trail.



2. Model training and offline evaluation

Repeatable pipeline

The training entry point reads versioned JSON configuration, loads the raw data, computes shared features, and creates an 80/20 stratified split using random seed 42. Each model is an sklearn Pipeline combining a ColumnTransformer with the estimator. Numeric columns receive median imputation and standardization. Categorical columns receive most-frequent imputation and one-hot encoding with unknown-category tolerance. The fitted artifact includes the entire preprocessing pipeline, decision threshold, model name, and model version; this removes manual preprocessing at deployment.

Baseline and candidate

The baseline is class-balanced logistic regression, chosen because it is fast, interpretable, and sets a credible linear benchmark. The candidate is a class-balanced random forest with 300 trees, maximum depth eight, and minimum leaf size six. These constraints reduce variance and discourage the model from memorizing sparse one-hot combinations. The comparison is intentionally small: assignment value comes from a defensible evaluation and lifecycle, not broad hyperparameter search.

Metric choice

ROC-AUC is the primary promotion metric because retention operations rank customers before selecting a capacity-dependent threshold. It evaluates ordering quality across all thresholds and is less misleading than accuracy on the imbalanced label. Accuracy is reported for accessibility, while precision measures incentive efficiency, recall measures coverage of actual churners, and F1 summarizes their balance. The confusion matrix exposes the operational counts at the 0.50 threshold.

Model	ROC-AUC	Accuracy	Recall	F1
Logistic baseline	0.8456	0.7402	0.7995	0.6203
Random forest candidate	0.8457	0.7644	0.7754	0.6360

Promotion decision

A candidate is promoted only if its ROC-AUC is at least 0.80 and it is no more than 0.01 below the baseline. The random forest passed: its ROC-AUC was 0.0001 higher, accuracy improved by 0.0242, precision improved from 0.5068 to 0.5390, and F1 improved by 0.0157. Recall declined from 0.7995 to 0.7754, a cost to track when choosing a business threshold. On 1,409 test cases, the promoted model produced 787 true negatives, 248 false positives, 84 false negatives, and 290 true positives.

Artifacts and reproducibility

The trained pipeline is saved as models/model.joblib; reference feature statistics are saved alongside it; and evaluation.json records data counts, class balance, all metrics, confusion matrices, guardrails, and the selected model. requirements.txt pins the environment and configs/model.json pins random state and estimator choices. The raw CSV hash is documented in README.md. These controls make another reviewer able to reproduce the result and distinguish code, configuration, source data, and generated evidence.

Limits of offline evidence

This is one random holdout from a static sample. It does not measure temporal generalization, campaign lift, calibration, fairness across protected or proxy groups, or the causal effect of retention offers. A production gate would add a time-based test, probability calibration, segment-level error analysis, and a shadow or A/B phase before business use.

3. Serving and inference pattern

Online request-response choice

Online scoring supports a service agent or digital workflow while a customer session is active. Model latency should remain below roughly 100 ms, leaving room for network and application overhead. Batch scoring remains useful for scheduled campaigns, but the API demonstrates the stricter contract.

API contract

FastAPI exposes GET /health and POST /predict. Pydantic validates fields and ranges. Responses include class, probability, and model_version for downstream thresholding, audit, and rollback. The model is cached after first load and preprocessing remains inside the serialized pipeline.

FastAPI online prediction

\$

```
POST /predict HTTP/1.1
```

```
content-type: application/json
```

```
{ "tenure": 1, "Contract": "Month-to-month", ... }
```

```
HTTP/1.1 200 OK
```

```
{ "prediction": 1, "churn_probability": 0.6842,  
  "model_version": "churn-rf-1.0.0" }
```

Performance measurement

A 250-request TestClient benchmark measured 39.994 ms average latency, 34.588 ms p95, and 25.0 requests/second. A few cold-start or scheduling outliers raised the mean above p95, so production testing should separate cold and warm traffic. This local result is feasibility evidence, not a capacity guarantee.

Deployment, security and scaling

The Dockerfile starts Uvicorn on port 8000 with pinned dependencies. Production would load a checksum-verified registry artifact, use replicated workers behind TLS and authentication, and autoscale on traffic and latency. Logs should contain request IDs rather than raw profiles. Scores guide outreach, not punitive action; fairness, retention limits, and intended-use controls remain required.

4. Monitoring, data quality and retraining

Monitoring plan

Three layers are monitored. Infrastructure metrics include request count, average and p95 latency, HTTP 4xx/5xx rate, model-load failures, CPU, and memory. Data metrics include row counts, schema compatibility, missing values, invalid ranges, unseen categories, and shifts in numeric means. Model and business metrics include score distribution, predicted churn rate, precision/recall/ROC-AUC once labels mature, outreach acceptance, retained revenue, and incentive cost. Operations owns service alerts, data engineering owns schema and freshness, and the ML owner owns drift and performance alerts.

Working quality and drift check

The implemented check reads a recent batch, applies the shared feature module, and compares MonthlyCharges, TotalCharges, tenure, and avg_charge_per_month with training reference statistics. It records missing rates and a standardized mean shift. A warning fires when missingness exceeds 5% or the mean shifts by more than one training standard deviation. The supplied demonstration batch matches the source distribution, so it produced status ok over 7,043 rows. This lightweight signal is understandable but does not detect changes in variance, categories, or feature relationships; PSI or a two-sample test would strengthen it.

Monitoring and performance evidence

\$

```
drift_check recent_batch.csv
status: ok | rows_checked: 7043
warnings: []

benchmark /predict
requests: 250
average_latency_ms: 39.994
p95_latency_ms: 34.588
throughput_requests_per_second: 25.0
```

Alerts and dashboards

A serving dashboard would show traffic, latency percentiles, error rate, prediction distribution, and model version. A daily data dashboard would show freshness, volume, nulls, categorical cardinality, and drift by feature. Warning thresholds could be p95 above 100 ms for 10 minutes, 5xx above 1%, input volume below 70% of its weekday baseline, or any critical field missing above 5%. Model-performance alerts should wait for enough labeled outcomes to avoid reacting to noise.

Retraining policy

The included function returns retrain=true when any of three signals fires: at least 30 new days of data are available; recent ROC-AUC falls more than 0.03 below the approved baseline; or maximum standardized mean drift exceeds 1.0. A real orchestrator would require data-quality checks first, train a challenger, evaluate the same promotion policy, and require approval before replacing the artifact. Retraining is therefore a trigger for evaluation, not automatic promotion.

Incident scenario: upstream schema change

Suppose the billing system renames MonthlyCharges or starts sending currency-formatted strings. API validation would reject malformed online records, the error-rate alert would rise, and batch quality checks would fail before training. The on-call engineer would stop the affected ingestion, preserve the last known-good model, and examine the producer contract. If backward-compatible parsing is safe, it would be added with a regression test; otherwise the producer would roll back. Retraining occurs only after corrected data passes validation. If a bad artifact was deployed, model_version enables rapid rollback.

5. Trade-offs, limitations and future work

Key engineering trade-offs

- Online scoring is fresh but creates availability and latency obligations compared with nightly batch scoring.
- The forest improves several metrics and captures nonlinear effects; logistic regression is faster and easier to explain. Their nearly equal ROC-AUC makes the choice uncertain.
- Shared code reduces skew, but a feature platform would also enforce point-in-time correctness, ownership, and lineage.
- Mean-shift checks are transparent but miss multivariate drift and cannot prove degradation without labels.
- The reproducible 0.50 threshold is not business-optimal; production should minimize churn loss plus outreach costs under capacity.

Limitations

The static, untimestamped dataset makes random splitting optimistic. It lacks interaction, complaint, network-quality, and offer-history data. The implementation uses a local artifact, synchronous scoring, local load testing, JSON monitoring, and no connected feedback loop.

Future work

Next steps are time-based validation, calibrated probabilities, an economic threshold, segment fairness checks, and a registry storing checksum, metrics, data version, approval, and rollback state. Monitoring should add categorical and distribution drift plus delayed outcome quality. Shadow deployment and an A/B test should establish retention lift.

Verification evidence

```
Automated verification
$
PYTHONPATH=src .venv/bin/pytest -q

tests/test_api.py PASSED
tests/test_features.py PASSED
tests/test_ingestion_monitoring.py PASSED

4 passed in 3.18s
```

Conclusion

The project supplies working evidence across data, training, serving, and monitoring. Its strongest choice is one feature path inside the serialized pipeline, supported by explicit promotion and retraining rules. ROC-AUC 0.8457 is suitable for demonstration; production value still requires temporal validation, economic thresholding, safe deployment, and labeled feedback.

References

- IBM. Telco Customer Churn sample dataset. <https://github.com/IBM/telco-customer-churn-on-icp4d>
- scikit-learn documentation. <https://scikit-learn.org/stable/>
- FastAPI documentation. <https://fastapi.tiangolo.com/>